

1 begin

2 using HypertextLiteral

3 using PlutoUI

4 using PlutoTeachingTools

5 using PlutoExtras

6 using PlutoTest

7 using ShortCodes

8 end

1 sec

1 using PlutoLinks

3.5 sec

1 using PlutoSlides

77.2 ms

1 import MarkdownLiteral: @markdown

456 ms

beamer_color = "#ff7f50"

1 beamer_color = "#ff7f50"

32.7 μs

1 PlutoSlides.slide_mode_settings(footer_left="David Métivier", footer_center="Julia for scientific computing and research", color_subtitle_bg=beamer_color, h3_title=true,

2 pdf_aspect="a4", pdf_stretch=0.80, #1.0495,

3 font_family=19, font_size=19,

4 max_width="98%"

5)

1.6 sec

Slide Mode

1 slide_mode_button()

32.6 μs

1 # ExtendedTableOfContents(hide_preamble = false)

40.2 μs

Julia: a language made for scientific computing and research

David Métivier

Chargé de recherche INRAE - Montpellier

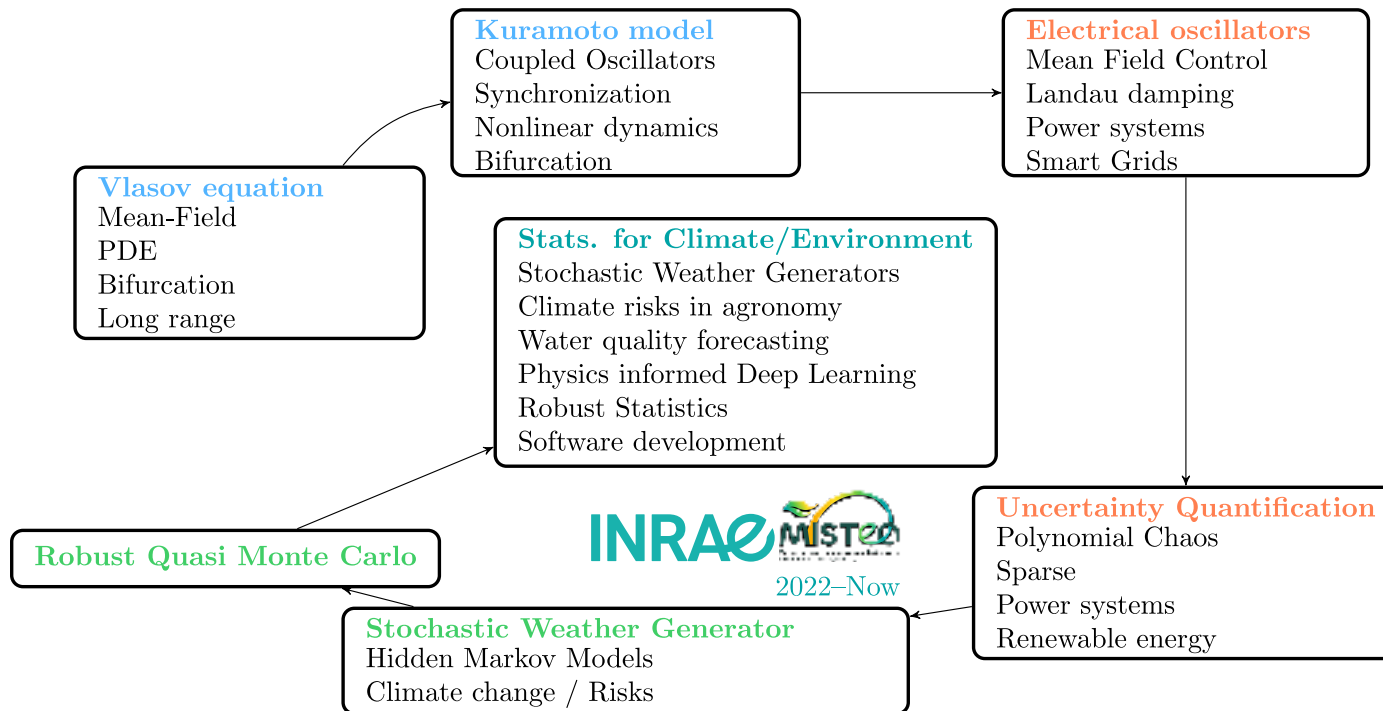


[credit]:

This talk was partly inspired by [Guillaume Dalle Julia talk](#) + [Valentin Churavy](#) + [Tim Holy lectures](#) + videos and link shown in the notebook

How I Started Using Julia

Ph.D. *Laboratoire J.A. Dieudonné*
2014-2017 with Julien Barré



Post-Doc *Centre de Mathématiques Appliquées*
2020-2022 with Emmanuel Gobet

Post-Doc *Los Alamos National Laboratory*
2017-2020 with Michael Chertkov

Coding experience

- Matlab/Mathematica (Internship)
- C (with GPU) with Gnuplot/xmgrace (School and PhD)
- Very few Python at (School and PhD)
- Julia (Los Alamos Postdoc 1 2018)

Package

- ExpectationMaximization.jl (*Developer*)
- QuasiMonteCarlo.jl (**Contributor**)
- StochasticWeatherGenerator.jl (*Developer*)
- PlutoSlides.jl (*Developer*)
- Smaller WIP projects RobustMeans.jl + ABCMethods.jl + PeriodicHiddenMarkovModels.jl (*Developer*)
- small contributions

Goal of the presentation

Program

- An overview of Julia's main features, functionalities, tools, and packages
- This is **NOT** a step-by-step tutorial; see the Resource section at the end for some links.
- You can follow along: Notebooks are available

Goal: Give you a feeling that this is a **great modern tool for complex scientific problems** and teaching, and that **the language favors collaborative work**.

Disclaimer

I will present many things: **a lot already exists in other languages**, sometimes as add-ons or wrapped C/C++ libraries.

Julia language and most of its packages are inspired by many cool existing things.

In this talk I want to emphasize the simplicity of some features and how far they can take us.

The Julia story

Thoughts by Gaël Varoquaux in May 2010 (co-founder of scikit-learn)



Gaël Varoquaux

Thu 20 May 2010

[←Home](#)

Personal views on scientific computing

under [science](#) [academia](#) [scientific computing](#) [selected](#) [scientific software](#) Read time: 3 min.

My contributions to the scientific computing software ecosystem are motivated by my vision on computational science.

Scientific research relies more and more on computing. However, most of the researchers are not software engineers, and as computing is becoming ubiquitous, the limiting factor becomes more and more the **human factor** [G. Wilson, 2006] [P. Norvig, 2009].

To address the needs of computing accross scientific fields, I believe that we need a **general-purpose, high-level, interactive, and highly-readable** language and set of tools for scientific computing.

My take

I would add **speed/performance** to these requirements

Why Julia was created?

“ We are power **Matlab** users. Some of us are **Lisp** hackers. Some are **Pythonistas**, others **Rubyists**, still others **Perl** hackers. There are those of us who used **Mathematica** before we could grow facial hair. There are those who still can't grow facial hair. We've generated more **R** plots than any sane person should. **C** is our desert island programming language. ”

We love all of these languages; they are wonderful and powerful. For the work we do — **scientific computing, machine learning, data mining, large-scale linear algebra, distributed and parallel computing** — each one is perfect for some aspects of the work and terrible for others. **Each one is a trade-off.**

We are greedy: we want more.

Why we created Julia – Jeff Bezanson, Stefan Karpinski, Viral B. Shah, and Alan Edelman (2012)

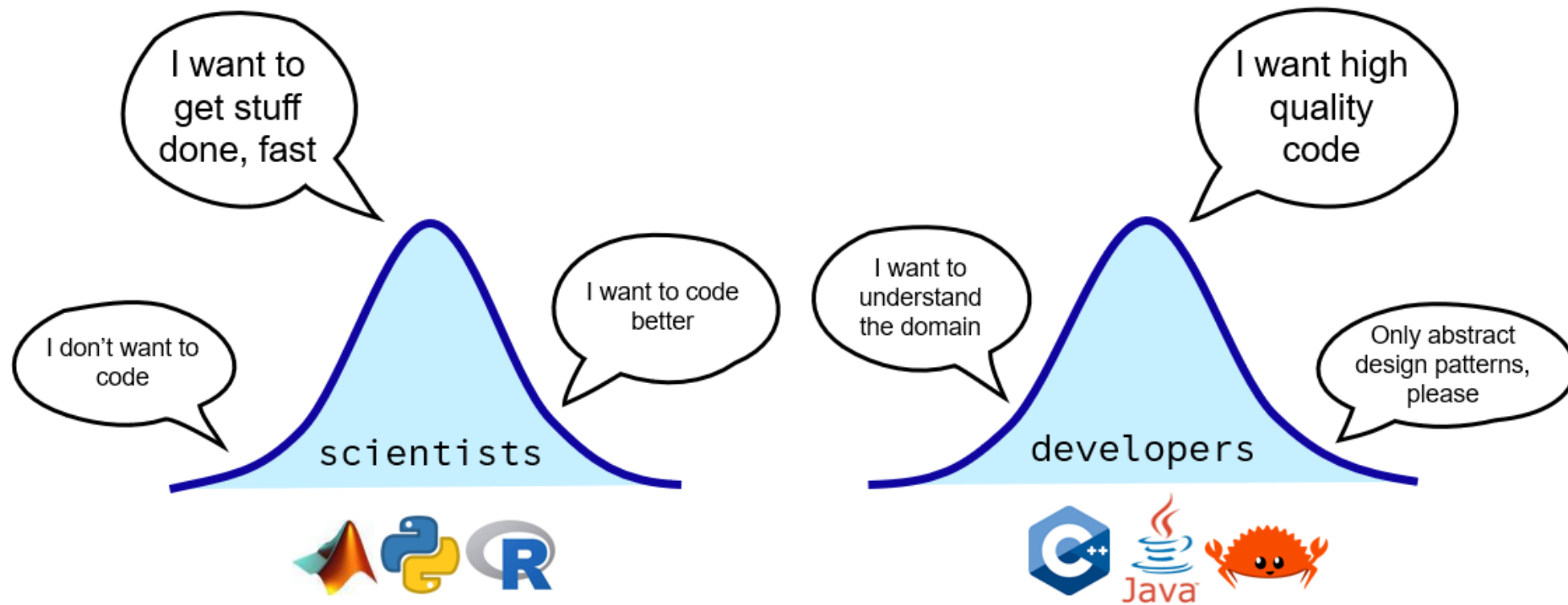
“ Julia is designed to be easy and fast and questions notions generally held to be “laws of nature” by practitioners of numerical computing: ”

1. High-level dynamic programs have to be slow.
2. **One must prototype in one language and then rewrite in another language for speed or deployment.**
3. There are parts of a system appropriate for the programmer, and other parts that are best left untouched as they have been built by the experts.

Julia: A Fresh Approach to Numerical Computing  – Jeff Bezanson, Alan Edelman, Stefan Karpinski, and Viral B. Shah (2017)

The Two Languages Problem

Scientists vs Developers (1/2)



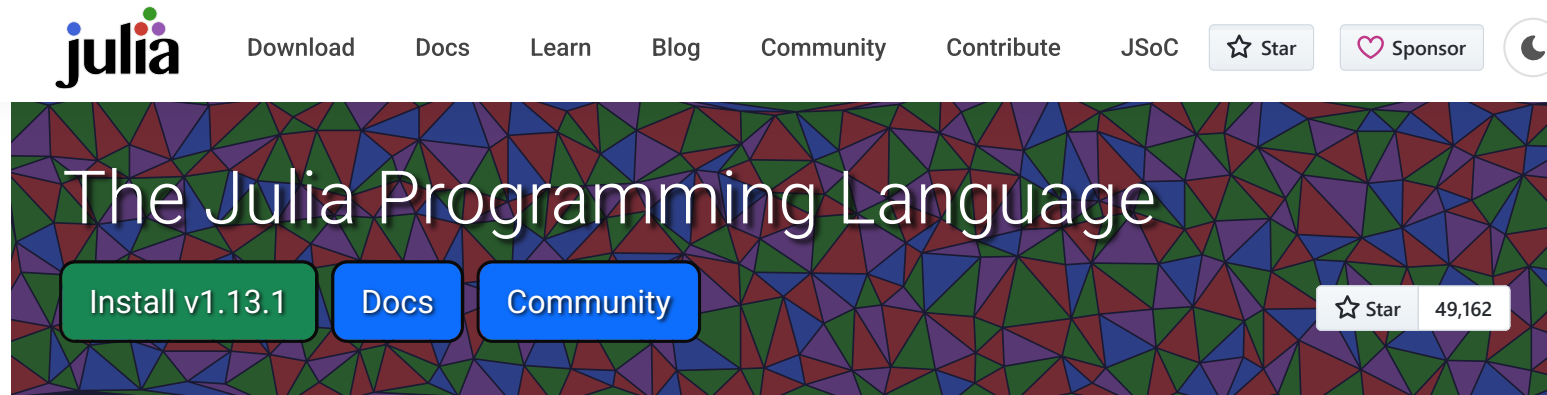
From [My Target Audience](#) by Matthijs Cox (2023)

Scientists vs Developers (2/2)



From My Target Audience by Matthijs Cox (2023)

Get started



Fast

Julia was designed for [high performance](#). Julia programs automatically compile to efficient native code via LLVM, and support [multiple platforms](#).

Composable

Julia uses [multiple dispatch](#) as a paradigm, making it easy to express many object-oriented and [functional](#) programming patterns. The talk on the [Unreasonable Effectiveness of Multiple Dispatch](#) explains why it works so well.

Dynamic

Julia is [dynamically typed](#), feels like a scripting language, and has good support for [interactive](#) use, but can also optionally be separately compiled.

General

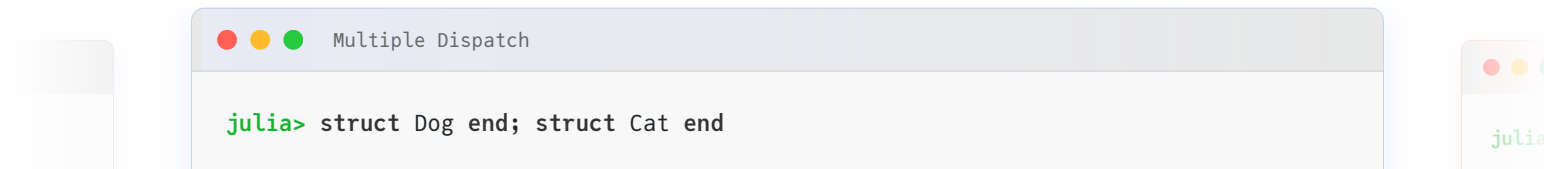
Julia provides [asynchronous I/O](#), [metaprogramming](#), [debugging](#), [logging](#), [profiling](#), a [package manager](#), and the ability to [build binaries](#).

Reproducible

[Reproducible environments](#) make it possible to recreate the same Julia environment every time, across platforms, with [pre-built binaries](#).

Open source

Julia is an open source project with over 1,000 contributors. It is made available under the [MIT license](#). The [source code](#) is available on GitHub. Julia has a [welcoming community](#) accessible to all backgrounds.



Syntax

One-line function definitions and *L^AT_EX* symbols

φ_1 (generic function with 1 method)

```
1  $\varphi_1(x) = \exp(-x^2) / \sqrt{2\pi}$ 
```

769 μ s

$\text{cat} = 1$

```
1  $\text{cat} = 1$ 
```

34.6 μ s

2

```
1  $\text{cat} + \text{cat}$ 
```

38.9 μ s

λ (generic function with 1 method)

```
1 function  $\lambda(x,y)$ 
2     1+1
3 end
```

595 μ s

cartesian (generic function with 2 methods)

```
1 cartesian(r,  $\theta$ ,  $\phi = 0$ ) = r*sin( $\theta$ )*cos( $\phi$ ), r*sin( $\theta$ )*sin( $\phi$ ), r*cos( $\theta$ )
```

1.1 ms

Not just good looking: improve readability!

false

```
1  $2 \in 4:7$  # returns 'false'
```

17.8 μ s

true

```
1  $[1,5] \subseteq 1:5$  # returns 'true'
```

19.5 μ s

f (generic function with 1 method)

```
1  $f(x) = x[1]^2 + x[2]^2$ 
```

1.6 ms

∇f (generic function with 1 method)

```
1  $\nabla f(x) = [2x[1], 2x[2]]$ 
```

1.6 ms

Built-in arrays

⌵ `A = 2×2 Matrix{Int64}:`

```
 1  1  
 1  0
```

```
1 A = [1 1;  
2      1 0]
```

⋮

▶ 56.1 μs

⌵ `2×2 Matrix{Int64}:`

```
89  55  
55  34
```

```
1 A^10
```

⋮

▶ 68.3 μs

Broadcast

- Vectorization of operators and functions just with dot.
 - It is a language feature
 - It works with every functions
- Vectorization is not **naturally** faster than loops but might be more convenient sometimes.

▶ [2, 3, 4]

1 [1, 2, 3] .+ 1

▶ 104 ms

▶ [2.71828, 7.38906, 20.0855]

1 exp.([1, 2, 3])

▶ 81.2 ms

▶ [0.146763, 0.00730688, 4.92334e-5]

1 φ_1 .([1, 2, 3])

▶ 76.6 ms

▶ [1 ⇒ "a", 2 ⇒ "b", 3 ⇒ "c"]

1 [1, 2, 3] .=> ["a", "b", "c"]

▶ 145 ms

Distributions

```
dist = Distributions.Normal{Float64}(μ=0.0, σ=1.0)
1 dist = Normal()
+ 35.9 μs
data =
▶ [1.52037, 2.31588, -1.6641, 0.333765, -1.17785, -0.167483, 0.950861, 0.0483696, -0.82446, -1.04135, 0.465383, -1.52251, 0.293616, -0.490347, 0.0940785, -0.0820523, 0.2
1 data = rand(dist, 100)
+ 56.3 μs
```

Error message

The backend must not support the series type Val{:shape}, and there isn't a series recipe defined.

Show stack trace...

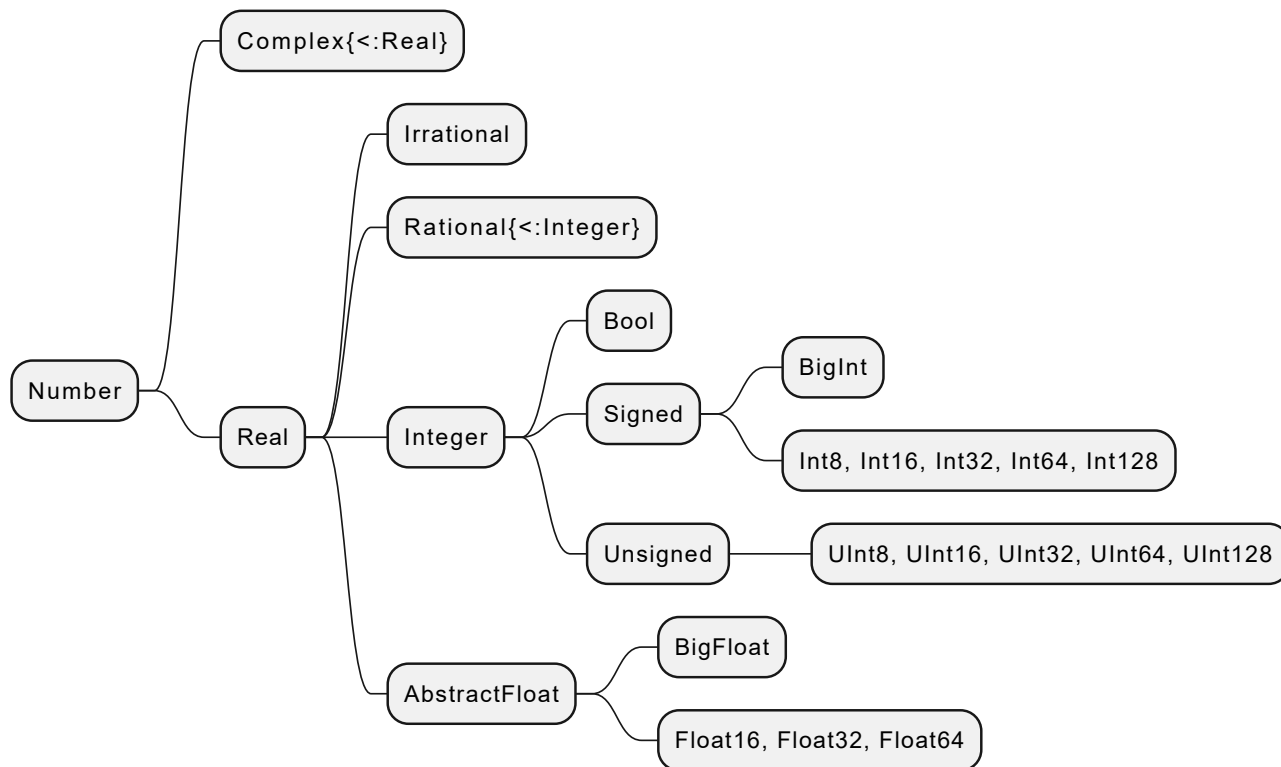
```
1 histogram(data, label = dist, ylabel = "PDF", size = (400, 300))
+ ---
1.09142539539915
1 rand(dist) + 1
+ 46.5 μs
▶ [0.614474]
1 rand(dist, 1)
+ 41.6 μs
```

Question:

Why does it return an error?

Types

Number hierarchy



```

abstract type Number end
abstract type Real <: Number end
abstract type AbstractFloat <: Real end
abstract type Integer <: Real end
abstract type Signed <: Integer end
abstract type Unsigned <: Integer end
  
```

[Ref]:

See documentation on Types